

Chapter 19. Spreadsheet Automation: Macros, VBA, Office Scripts & Apps Script

Record it, rewrite it properly, make it fast, and hand it over safely.

Approved chapter v1

Approved 18 September 2026

Data and expected results checked with Python

Contents

Chapter 19. Spreadsheet Automation: Macros, VBA, Office Scripts & Google Apps Script

- Why this matters
- In plain English
- 19.1 When automating inside a spreadsheet is the right answer
- 19.2 Recording your first macro
- 19.3 The VBA editor
- 19.4 Programming from zero, in VBA
- 19.5 Excel's object model
- 19.6 The everyday toolkit
- 19.7 Sending the report by Outlook
- 19.8 Custom functions and a simple form
- 19.9 Debugging and error handling
- 19.10 Making macros fast
- 19.11 Office Scripts: Excel on the web
- 19.12 Google Sheets: macros and Apps Script
- 19.13 Triggers, email, and APIs in Apps Script
- 19.14 The same job in three languages
- 19.15 Living with macros responsibly
- Common mistakes and how to spot them
- In the real world: the macro that ran for nine years
- Tools
- The project: two automations, end to end
- Timed challenge: forty-five minutes in the editor
- You've got it when...
- Recap
- Practice exercises
- Key terms
- Where this leads
- Answers to practice exercises

Chapter 19. Spreadsheet Automation: Macros, VBA, Office Scripts & Google Apps Script

Part II — The Analyst

Chapter at a glance

You will learn to: decide when automating inside a spreadsheet is the right answer · record a macro, save a macro-enabled workbook, and handle macro security without turning it off · read recorded code and rewrite it properly · program from zero in VBA: variables, `If`, `Select Case`, loops, arrays, collections, `Sub` and `Function` · work with Excel's object model instead of clicking · build the everyday toolkit: last row, loop the sheets, consolidate a folder of files, clean, format, pivot, PDF · send an Outlook email with the report attached or in the body · write custom worksheet functions and a simple form · debug with breakpoints and the Immediate window, and handle errors on purpose · make a macro fast with arrays and screen updating · do the same work in **Office Scripts** (Excel on the web, TypeScript) and **Google Apps Script** (Sheets, JavaScript), including triggers, HTML email, and API calls · keep macros maintainable, and know when to move the job to Power Query, Python, or a pipeline.

Before you start: Chapters 10 and 11 (spreadsheets, Power Query), Chapter 14 (cleaning), Chapter 15 (charts). **No programming experience is assumed:** section 19.4 teaches the basics from zero in VBA, so this chapter stands on its own before the Python block.

Time needed: 20–25 hours, spread over three weeks.

Tools: **Excel for Windows** (Microsoft 365 or 2016+) for VBA and Outlook automation; **Excel on the web** with a Microsoft 365 business plan for Office Scripts; a **Google account** for Sheets and Apps Script. Excel for Mac runs VBA but not the Outlook or Windows-specific parts.

Practice data: `companion/ch19/branch_files/` — twelve workbooks (four branch sales offices × October, November, December 2025), built from Chapter 14's clean Q4 data, plus `_notes.txt` so your folder code has to filter by extension. `expected_results.md` lists every number a correct consolidation must produce. `enquiries_sample.csv` is the Google Form data for the Apps Script project.

Why this matters

Walk into most Indian offices and the real reporting engine isn't a BI tool. It's a workbook with a button on it, written years ago by someone who has since left.

That's not a failure of taste. Spreadsheets are where the data already is, where the finance team is comfortable, and where a change takes minutes instead of a change request. A macro that consolidates twelve branch files and emails a PDF at 9 a.m. is worth more to the business than an elegant pipeline nobody funded.

It's also where the most common analyst horror stories live: the macro that only runs on one laptop, the one that silently skips a file, the one with a password in line 40, the one nobody can read. This chapter teaches both halves: how to automate a spreadsheet properly, and how to keep the result from becoming a liability.

There's a career angle too. "Knows VBA" still appears in analyst job descriptions across manufacturing, distribution, and finance in India, and the first automation you build at work is usually a spreadsheet one, because it needs no permission and no new software. Getting it right is how you earn the room to do the bigger things in Chapters 18 and 20.

In plain English

A spreadsheet is a workshop bench. You do the job by hand: open the files, copy, paste, sort, format, save, attach, send.

A **macro** is a jig. You set it up once, and every repeat of the job clamps into the same place and comes out the same way. The first one you make is a **recording**: you do the job once with the recorder on, and the spreadsheet writes down every movement, including the clumsy ones.

Recordings are literal. If you clicked cell B7 because that's where last month's data ended, the recording clicks B7 forever, even when this month has more rows. So the second step is always the same: read what the recorder wrote, keep the useful parts, and replace the literal clicks with instructions that describe what you meant. "Go to the last row with data", not "go to B7". "For every file in the folder", not "open this one file".

That's the whole chapter. A recorder to get started, a language (VBA, TypeScript, or JavaScript) to say what you meant, and a set of habits so the jig still works when someone else picks it up.

19.1 When automating inside a spreadsheet is the right answer

Situation	Best tool
The data already lives in workbooks, and the output must be a workbook people edit	VBA or Office Scripts
A repetitive shaping job: combine files, unpivot, clean columns	Power Query (Chapter 11), not a macro
A report several people read, refreshed on a schedule	Power BI (Chapter 16)
Data from a database or an API, or anything statistical	Python (Chapter 18)
A Google Sheets workflow: forms, emails, triggers	Apps Script
Formatting, saving as PDF, sending Outlook email, driving Excel itself	VBA
A pipeline with dependencies, retries, and monitoring	An orchestrator (Chapter 46)

Two rules of thumb:

1. **If Power Query can do it, use Power Query.** It's recorded, refreshable, and doesn't need macro-enabled files or trust settings. Most "combine these files" macros written before 2015 would be three clicks today.
2. **Automate the boring shell, not the thinking.** Macros are excellent at opening, copying, formatting, saving, and sending. They're a poor place for business logic that changes often, and a terrible place for anything you can't explain to the person who inherits it.

What a macro is, and what .xlsm means

A **macro** is code stored inside a workbook (or in your personal macro workbook) that Excel can run. Because code in a file can do anything your account can do, Excel separates file types:

Extension	Holds macros?	Use
.xlsx	No	Normal workbooks
.xlsm	Yes	Macro-enabled workbooks
.xlsb	Yes	Binary format: faster to open for very large files
.xltm	Yes	Macro-enabled template

Save a workbook containing code as .xlsx and Excel throws the code away, with one warning that everybody clicks past. It's the most common way a morning's work disappears.

Macro security, without switching it off

File → Options → Trust Center → Trust Center Settings → Macro Settings offers four choices. The right one is the default: **Disable VBA macros with notification**, which lets you enable a file you trust after seeing the banner.

Two things to know in a company setting:

- **Files from the internet or email are blocked**, with a red banner rather than the yellow "Enable content" one. Since 2022 Microsoft blocks macros in files marked with the Mark of the Web by default. The fix is **not** to unblock every file: it's to put shared macro workbooks in a **trusted location** (**Trust Center → Trusted Locations**) on a network path your IT team approves.
- **Never tell colleagues to enable all macros.** That setting is how ransomware arrives. Trusted locations and, in larger companies, signed macros are the grown-up answers.

19.2 Recording your first macro

The recorder is the fastest way to learn the object model: do the thing, then read what Excel wrote.

1. **View → Macros → Record Macro** (or **Developer → Record Macro**; enable the Developer tab in **File → Options → Customize Ribbon**).
2. Name it without spaces (`Format_Sales_Sheet`), optionally give it a shortcut, and choose where to store it:
 - **This Workbook:** the macro travels with the file.
 - **Personal Macro Workbook:** a hidden workbook (`PERSONAL.XLSB`) that opens with Excel, so the macro is available in every file on your machine. This is where your personal cleanup tools belong.
3. Do the job once, carefully.
4. **Stop Recording**, then **Alt+F8** to run it, or **Alt+F11** to read it.

Attach it to a button so other people can run it: **Developer → Insert → Form Controls → Button**, draw it, assign the macro. (Form controls, not ActiveX; they're simpler and survive being emailed.)

What the recorder gives you

Recording "select the data, make the header bold, freeze the top row, autofit" produces something like this:

```
Sub Format_Sales_Sheet()
'
' Format_Sales_Sheet Macro
'
    Range("A1:K1").Select
    Selection.Font.Bold = True
    Rows("2:2").Select
    ActiveWindow.FreezePanes = True
    Cells.Select
    Cells.EntireColumn.AutoFit
    Range("A1").Select
End Sub
```

It works, and it has three problems that every recorded macro has:

- **It selects things.** Selecting is what a human does because a human needs to see the cell. Code doesn't.
- **It's literal.** `A1:K1` is this sheet's header today; next month's file has twelve columns.
- **It depends on what's active.** `ActiveWindow` and `Cells` mean "whatever is in front of me", so running it with the wrong sheet in view formats the wrong thing.

The rewritten version says what you meant:

```
Sub FormatSalesSheet(ws As Worksheet)
    With ws
        .Rows(1).Font.Bold = True
        .Activate                ' FreezePanes needs the sheet visible
        .Range("A2").Select
        ActiveWindow.FreezePanes = True
        .Cells.EntireColumn.AutoFit
    End With
End Sub
```

Now it takes the sheet as an argument, touches only that sheet, and works whatever the width of the data. (`FreezePanes` is one of the few things that really does need a selection, which is worth knowing so you don't hunt for a cleaner way.)

Watch out: the recorder can't record everything. Loops, conditions, folder handling, and error handling never appear in a recording, because you can't click them. The recorder teaches you object names; section 19.4 onward teaches you the rest.

19.3 The VBA editor

Alt+F11 opens the Visual Basic Editor (VBE). Four panes matter:

- **Project Explorer (Ctrl+R):** the tree of open workbooks, their sheets, `ThisWorkbook`, and **Modules**. Your code goes in a module (**Insert → Module**), not in a sheet, unless it's an event handler.
- **Code window:** where you type. Two drop-downs at the top navigate objects and their events.
- **Immediate window (Ctrl+G):** a REPL. `?ActiveSheet.Name` prints the answer; `Debug.Print` writes here.
- **Locals and Watch windows:** variable values while the code is paused.

Set two options once, in **Tools → Options**:

- **Require Variable Declaration** (ticked). It puts `Option Explicit` at the top of every new module, which forces you to declare variables and catches every typo. Without it, a misspelled variable is silently a new empty one, which is the single most common source of wrong answers in VBA.
- **Auto Syntax Check** (unticked, if the pop-ups annoy you; the red text still marks errors).

19.4 Programming from zero, in VBA

This section assumes you have never written code. If you've read Chapter 17, the ideas are the same and only the spelling changes.

Statements, comments, and structure

Code lives in a **procedure**. There are two kinds: a **Sub** does something, a **Function** returns a value.

```
Option Explicit          ' every variable must be declared

Sub SayHello()
    ' a comment starts with an apostrophe
    MsgBox "Hello from Riverstone"    ' shows a dialog box
    Debug.Print "this goes to the Immediate window"
End Sub
```

Run it with **F5** in the editor, or **Alt+F8** from Excel. Use `Debug.Print` while developing, not `MsgBox`: a message box in a loop of 12 files means 12 clicks.

Variables and types

A **variable** is a name for a value. `Dim` declares it, and the type says what it can hold.

```
Sub VariableDemo()
    Dim branch As String
    Dim rowCount As Long
    Dim netRevenue As Double
    Dim isFinal As Boolean
    Dim orderDate As Date

    branch = "Mumbai H0"
    rowCount = 25832
    netRevenue = 423872808#
    isFinal = True
    orderDate = DateSerial(2025, 12, 31)

    Debug.Print branch & ": " & Format(rowCount, "#,##0") & " rows, " & Format(netRevenue, "#,##0.00")
    Debug.Print "Final? " & isFinal & "; period ends " & Format(orderDate, "dd mmm yyyy")
End Sub
```

Type	Holds	Notes
String	Text	Join with & (not +)
Long	Whole numbers	Use instead of Integer, which overflows above 32,767: there are more than 32,767 rows in most files
Double	Decimals	The usual numeric type
Boolean	True / False	
Date	Dates and times	DateSerial(y, m, d), Now, Date
Variant	Anything	The default when you don't say; flexible, slower, and hides mistakes
Object types	Workbook, Worksheet, Range	Assign with Set

Two VBA-specific rules that catch everyone:

- **Objects need Set**: `Set ws = ThisWorkbook.Worksheets("Sales")`, but `total = 0` for a plain value.
- **Option Explicit at the top of every module**. Without it, `netRevene = 100` creates a new variable and your total stays zero.

Conditions

```
Sub CheckTarget(actual As Double, target As Double)
    If actual >= target Then
        Debug.Print "Above target"
    ElseIf actual >= target * 0.9 Then
        Debug.Print "Close: " & Format(actual / target, "0.0%")
    Else
        Debug.Print "Below target by " & Format(target - actual, "#,##0")
    End If
End Sub

Sub BandBySize(value As Double)
    Select Case value
        Case Is >= 50000
            Debug.Print "very large"
        Case 25000 To 49999.99
            Debug.Print "large"
        Case 10000 To 24999.99
            Debug.Print "medium"
        Case Else
            Debug.Print "small"
    End Select
End Sub
```

Select Case is VBA's cleanest way to write a banding rule, and it reads better than five ElseIf s. Comparisons are =, <>, <, <=, >, >=, combined with And, Or, Not.

Loops

```
Sub LoopDemo()
    Dim i As Long, ws As Worksheet, total As Double

    For i = 1 To 5                                ' counted loop
        Debug.Print "row " & i
    Next i

    For Each ws In ThisWorkbook.Worksheets        ' loop a collection
        Debug.Print ws.Name & " has " & ws.UsedRange.Rows.Count & " rows"
    Next ws

    i = 1
    Do While ThisWorkbook.Worksheets(1).Cells(i, 1).Value <> "" ' loop until a condition fails
        total = total + ThisWorkbook.Worksheets(1).Cells(i, 1).Value
        i = i + 1
    Loop
    Debug.Print "total " & total
End Sub
```

- **For ... Next** when you know how many times (rows 2 to the last row).
- **For Each ... Next** over a collection (worksheets, cells in a range, files).
- **Do While / Do Until** when the end depends on the data. Always make sure something changes inside, or the loop never ends (**Ctrl+Break** stops a runaway macro).
- **Exit For and Exit Do** leave early, like Chapter 17's `break`.
- **Step -1** counts backwards, which is what you need when deleting rows: deleting row 5 moves row 6 up, so a forward loop skips it.

Arrays and collections

```
Sub ArrayDemo()
    Dim branches As Variant, i As Long
    branches = Array("Mumbai H0", "Bengaluru", "Delhi", "Kolkata")

    For i = LBound(branches) To UBound(branches)
        Debug.Print i & ": " & branches(i)
    Next i

    Dim targets As Object
    Set targets = CreateObject("Scripting.Dictionary") ' a lookup table
    targets("Mumbai H0") = 150000000#
    targets("Bengaluru") = 120000000#
    Debug.Print targets("Bengaluru"), targets.Exists("Delhi")
End Sub
```

A **Dictionary** is VBA's version of Chapter 17's dict and Chapter 14's mapping table: a key and a value, with `.Exists()` to check before you look up. It's the right tool for "map branch spelling to standard name" and for counting.

Procedures with arguments and return values

```
Function NetLine(quantity As Long, unitPrice As Double, Optional discountPct As Double = 0) As Double
    NetLine = quantity * unitPrice * (1 - discountPct / 100)
End Function

Sub UseIt()
    Debug.Print NetLine(45, 430, 5)      ' 18382.5
    Debug.Print NetLine(10, 290)        ' 2900
End Sub
```

A **Function** returns a value by assigning to its own name. **Optional** arguments have defaults. Arguments are passed **by reference** by default, meaning the procedure can change the caller's variable: write **ByVal** when you don't want that, which is most of the time.

Two habits from here on: **one procedure, one job**, and **no magic numbers** — put the folder path, the sheet name, and the thresholds in **Const** declarations at the top, where the next person can find them.

```
Private Const SOURCE_FOLDER As String = "C:\Riverstone\branch_files\"
Private Const MASTER_SHEET As String = "Master"
```

19.5 Excel's object model

VBA drives Excel by talking to **objects** arranged in a hierarchy: **Application** → **Workbook** → **Worksheet** → **Range**. Each has **properties** (things it is: `.Name`, `.Value`, `.Font.Bold`) and **methods** (things it does: `.Copy`, `.Delete`, `.SaveAs`).

```
Sub ObjectBasics()
    Dim wb As Workbook, ws As Worksheet, rng As Range

    Set wb = ThisWorkbook          ' the workbook holding this code
    Set ws = wb.Worksheets("Master") ' by name, not by position
    Set rng = ws.Range("A1:K1")

    ws.Range("A1").Value = "order_item_id"
    ws.Cells(2, 1).Value = 184000    ' Cells(row, column): easier in loops
    rng.Font.Bold = True
    Debug.Print ws.Name, ws.Cells(1, 1).Value, rng.Address
End Sub
```

The five things you'll write constantly

```
Sub RangeToolkit()
    Dim ws As Worksheet, lastRow As Long, lastCol As Long
    Set ws = ThisWorkbook.Worksheets("Master")

    lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row      ' last used row in column A
    lastCol = ws.Cells(1, ws.Columns.Count).End(xlToLeft).Column ' last used column in row 1
    Debug.Print lastRow, lastCol

    Dim data As Range
    Set data = ws.Range(ws.Cells(2, 1), ws.Cells(lastRow, lastCol)) ' the data without the header
    Debug.Print data.Address, data.Rows.Count

    ws.Range("A1").CurrentRegion.Select                        ' the contiguous block, like Ctrl+A
    ws.Rows(1).Insert                                         ' insert, delete, clear
    ws.Rows(1).Delete
    ws.Range("L:L").ClearContents
End Sub
```

`Cells(Rows.Count, "A").End(xlUp).Row` is the most useful line in VBA: it's "press Ctrl+Up from the bottom of column A", and it finds the last row whatever the size of this month's file. Use `UsedRange` only when you must; it can include formatted-but-empty cells and lie to you.

Avoid **Select** and **Activate**

Recorded code selects because you did. Written code shouldn't: it's slower, it depends on what's in front of the user, and it breaks when a dialog steals focus.

```
' recorded
Sheets("Master").Select
Range("A1").Select
Selection.Value = "Total"

' written
ThisWorkbook.Worksheets("Master").Range("A1").Value = "Total"
```

The same for `ActiveWorkbook` and `ActiveSheet` : use them only when you really do mean "whatever the user has open", and assign them to a variable straight away.

With , for readability

```
Sub FormatHeader(ws As Worksheet, lastCol As Long)
    With ws.Range(ws.Cells(1, 1), ws.Cells(1, lastCol))
        .Font.Bold = True
        .Font.Color = RGB(255, 255, 255)
        .Interior.Color = RGB(15, 92, 140)
        .HorizontalAlignment = xlCenter
        .EntireColumn.AutoFit
    End With
End Sub
```

`With` saves repetition and makes the block clearly about one thing. It's the closest VBA gets to a paragraph.

19.6 The everyday toolkit

Here is the work an analyst actually automates, in the order it usually happens. All of it runs against `companion/ch19/branch_files/` : twelve workbooks, four branch sales offices × three months of Q4 2025.

Consolidate every file in a folder

```
Option Explicit

Private Const SOURCE_FOLDER As String = "C:\Riverstone\branch_files\"
Private Const DATA_SHEET As String = "Sales"

Sub ConsolidateBranchFiles()
    Dim master As Worksheet, src As Workbook, srcWs As Worksheet
    Dim fileName As String, nextRow As Long, lastRow As Long, lastCol As Long
    Dim filesRead As Long

    Application.ScreenUpdating = False
    Application.DisplayAlerts = False

    Set master = ThisWorkbook.Worksheets("Master")
    master.Cells.ClearContents
    nextRow = 1

    fileName = Dir(SOURCE_FOLDER & "*.xlsx") ' Dir lists files one at a time
    Do While fileName <> ""
        Set src = Workbooks.Open(SOURCE_FOLDER & fileName, ReadOnly:=True)
        Set srcWs = src.Worksheets(DATA_SHEET)

        lastRow = srcWs.Cells(srcWs.Rows.Count, "A").End(xlUp).Row
        lastCol = srcWs.Cells(1, srcWs.Columns.Count).End(xlToLeft).Column

        If nextRow = 1 Then ' copy the header once, from the first file
            srcWs.Range(srcWs.Cells(1, 1), srcWs.Cells(1, lastCol)).Copy master.Cells(1, 1)
            master.Cells(1, lastCol + 1).Value = "source_file"
            nextRow = 2
        End If

        If lastRow >= 2 Then
            srcWs.Range(srcWs.Cells(2, 1), srcWs.Cells(lastRow, lastCol)).Copy master.Cells(nextRow, 1)
            master.Range(master.Cells(nextRow, lastCol + 1), _
                master.Cells(nextRow + lastRow - 2, lastCol + 1)).Value = fileName
            nextRow = nextRow + lastRow - 1
        End If

        src.Close SaveChanges:=False
        filesRead = filesRead + 1
        fileName = Dir ' Dir with no argument returns the next file
    Loop

    Application.DisplayAlerts = True
    Application.ScreenUpdating = True

    MsgBox filesRead & " files consolidated, " & Format(nextRow - 2, "#,##0") & " data rows.", vbInformation
End Sub
```

Points worth noticing, because they're the difference between a macro that works once and one that works every month:

- **Dir(path & "*.xlsx")** filters by extension, so `_notes.txt` is ignored. (`Dir` is old but everywhere; `FileSystemObject` via `CreateObject("Scripting.FileSystemObject")` is the richer alternative.)
- **The header is copied once**, from the first file, and the data starts at row 2 in every file after that.
- **A `source_file` column** records where each row came from. Without it, you can't trace a wrong number back, and you can't tell whether a file was consolidated twice.
- **`ReadOnly:=True`** means the macro can't accidentally modify a branch's file, and it opens even if someone else has it open.
- **`ScreenUpdating = False`** around the whole thing: on twelve files this is the difference between four seconds and forty.

On the companion files, this produces **25,832 data rows**. `expected_results.md` lists each file's row count so you can check the macro didn't skip one, and the complete set: **24,738 non-cancelled rows**, net revenue **₹423,872,808.00**, the same figure as Chapter 14's clean data.

Clean and standardize while you're there

```
Sub CleanMaster()  
    Dim ws As Worksheet, lastRow As Long, r As Long  
    Dim statusMap As Object  
  
    Set ws = ThisWorkbook.Worksheets("Master")  
    lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row  
  
    Set statusMap = CreateObject("Scripting.Dictionary")  
    statusMap("delivered") = "Delivered"  
    statusMap("dlvd") = "Delivered"  
    statusMap("cancelled") = "Cancelled"  
    statusMap("canceled") = "Cancelled"  
    statusMap("cxl") = "Cancelled"  
    statusMap("shipped") = "Shipped"  
    statusMap("pending") = "Pending"  
  
    For r = 2 To lastRow  
        Dim raw As String  
        raw = LCase(Trim(ws.Cells(r, 9).Value))      ' column I = status  
        If statusMap.Exists(raw) Then  
            ws.Cells(r, 9).Value = statusMap(raw)  
        Else  
            ws.Cells(r, 9).Interior.Color = RGB(255, 235, 200) ' flag, don't guess  
        End If  
        ws.Cells(r, 4).Value = Format(ws.Cells(r, 4).Value, "0000") ' customer_code keeps its zeros  
    Next r  
End Sub
```

This is Chapter 14's normalize-then-map, in VBA: lower-case and trim, look up in a dictionary, and **flag what isn't in the map** rather than passing it through. Section 19.10 shows how to do the same thing twenty times faster with an array.

Summarize, with a pivot table

```
Sub BuildSummaryPivot()  
    Dim ws As Worksheet, pvtWs As Worksheet, lastRow As Long, lastCol As Long  
    Dim cache As PivotCache, pvt As PivotTable  
  
    Set ws = ThisWorkbook.Worksheets("Master")  
    lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row  
    lastCol = ws.Cells(1, ws.Columns.Count).End(xlToLeft).Column  
  
    On Error Resume Next  
    ThisWorkbook.Worksheets("Summary").Delete      ' start clean; DisplayAlerts already off  
    On Error GoTo 0  
    Set pvtWs = ThisWorkbook.Worksheets.Add  
    pvtWs.Name = "Summary"  
  
    Set cache = ThisWorkbook.PivotCaches.Create( _  
        SourceType:=xlDatabase, _  
        SourceData:=ws.Range(ws.Cells(1, 1), ws.Cells(lastRow, lastCol)))  
    Set pvt = cache.CreatePivotTable(TableDestination:=pvtWs.Range("A3"), TableName:="SalesPivot")  
  
    With pvt  
        .PivotFields("source_file").Orientation = xlRowField  
        .PivotFields("status").Orientation = xlColumnField  
        .AddDataField .PivotFields("net_revenue"), "Net revenue", xlSum  
        .DataBodyRange.NumberFormat = "#,##0"  
    End With  
  
    pvtWs.Range("A1").Value = "Riverstone Q4 2025 – net revenue by file and status"  
    pvtWs.Range("A1").Font.Bold = True  
End Sub
```

Pivot tables in VBA are fiddly and worth it: the alternative is `SUMIFS` formulas that someone has to drag. If the summary needs to look a specific way every month, build it once by hand, record the formatting, and paste the useful parts in here.

Save as PDF

```
Function SaveSummaryAsPdf() As String
    Dim path As String
    path = ThisWorkbook.Path & Application.PathSeparator & _
        "Riverstone_Q4_2025_summary_" & Format(Date, "yyyy-mm-dd") & ".pdf"

    ThisWorkbook.Worksheets("Summary").ExportAsFixedFormat _
        Type:=xlTypePDF, FileName:=path, Quality:=xlQualityStandard, OpenAfterPublish:=False

    SaveSummaryAsPdf = path
End Function
```

`ExportAsFixedFormat` on a sheet exports that sheet; on the workbook, everything. Set the print area, orientation, and `FitToPagesWide` first (`ws.PageSetup...`), or the PDF arrives as nine pages of columns.

19.7 Sending the report by Outlook

```
Sub EmailSummary(pdfPath As String, netRevenue As Double, rowCount As Long)
    Dim outlookApp As Object, mail As Object
    Dim bodyHtml As String

    Set outlookApp = CreateObject("Outlook.Application") ' late binding: no reference needed
    Set mail = outlookApp.CreateItem(0) ' 0 = olMailItem

    bodyHtml = "<p>Good morning,</p>" & _
        "<p>Riverstone's Q4 2025 sales summary is attached.</p>" & _
        "<table style='border-collapse:collapse;font-family:Segoe UI,Arial;font-size:13px'>" & _
        "<tr><td style='padding:4px 12px;color:#5b6475'>Net revenue (non-cancelled)</td>" & _
        "<td style='padding:4px 12px;font-weight:bold'>" & Format(netRevenue, "₹#,##0.00") & "</td></tr>" & _
        "<tr><td style='padding:4px 12px;color:#5b6475'>Order lines consolidated</td>" & _
        "<td style='padding:4px 12px;font-weight:bold'>" & Format(rowCount, "#,##0") & "</td></tr>" & _
        "</table>" & _
        "<p style='color:#5b6475;font-size:12px'>Generated automatically from the twelve branch files on " & _
        Format(Now, "dd mmm yyyy HH:mm") & ".</p>"

    With mail
        .To = "sales.managers@riverstone.example"
        .Subject = "Riverstone Q4 2025 sales summary"
        .HTMLBody = bodyHtml
        .Attachments.Add pdfPath
        .Display ' .Send sends it immediately; .Display lets a human look first
    End With
End Sub
```

- **Late binding** (`CreateObject("Outlook.Application")`) avoids adding a reference to a specific Outlook version, so the file works on other machines.
- **.Display while developing, .Send when you trust it.** An automation that emails the wrong list is worse than no automation.
- **HTML in the body** beats an attachment nobody opens (Chapter 20 goes further). Keep it to inline styles and tables: Outlook's rendering engine ignores most modern CSS.
- **.CC , .BCC , .SentOnBehalfOfName , .Attachments.Add** cover the rest. For a shared mailbox, ask IT rather than storing someone's credentials.

Watch out: this drives the Outlook installed on your machine. It needs Outlook open (or it starts it), it won't run on a server without Outlook, and modern security tooling may prompt. For unattended sending, use the Microsoft 365 API route in Chapter 20 instead.

19.8 Custom functions and a simple form

A UDF: your own worksheet function

```
Public Function REBATEPCT(annualValue As Double) As Double
    ' Riverstone's loyalty rebate: 0% standard, 1% from 250,000, 2% from 400,000
    Select Case annualValue
        Case Is >= 400000: REBATEPCT = 2
        Case Is >= 250000: REBATEPCT = 1
        Case Else: REBATEPCT = 0
    End Select
End Function
```

Put it in a normal module and it appears in Excel as `=REBATEPCT(B2)` . UDFs are a good way to give the business one blessed version of a rule, instead of the same nested `IF` copied into forty workbooks.

Limits worth knowing: a UDF can read cells but shouldn't change them; it recalculates whenever Excel wants, so it must be fast; it isn't available in Excel on the web; and a workbook full of UDFs is slower than the same logic in Power Query or as a helper column. `Application.Volatile` forces recalculation on every change, and is almost always a mistake.

A UserForm, when a dialog is the better answer

For a macro that needs a choice ("which month?"), start with the built-in dialogs:

```
Sub AskForMonth()
    Dim answer As String
    answer = InputBox("Which month? (YYYY-MM)", "Riverstone consolidation", Format(Date, "yyyy-mm"))
    If answer = "" Then Exit Sub ' the user pressed Cancel
    If Not answer Like "####-##" Then
        MsgBox "Please use the format 2025-12.", vbExclamation
        Exit Sub
    End If
    Debug.Print "running for " & answer
End Sub

Sub PickFolder()
    With Application.FileDialog(msoFileDialogFolderPicker)
        .Title = "Choose the folder with the branch files"
        If .Show = -1 Then Debug.Print .SelectedItems(1)
    End With
End Sub
```

`InputBox` , `MsgBox` , and `Application.FileDialog` cover most needs and cost one line each. A **UserForm** (**Insert** → **UserForm** in the editor, then drag labels, text boxes, combo boxes, and buttons) is worth it when there are several inputs at once: a month, a branch, and a checkbox for "email it". Name the controls properly (`cmbBranch` , `txtMonth` , `btnRun`), write the code in the form's `btnRun_Click` event, and keep the actual work in a module the form calls, so the logic can be tested without the form.

19.9 Debugging and error handling

Debugging

Tool	How	Use
Breakpoint	Click the gray margin, or F9	Pause on a line
Step Into / Over / Out	F8 / Shift+F8 / Ctrl+Shift+F8	Walk through line by line
Immediate window	Ctrl+G , then <code>?expression</code>	Ask what a value is right now
Locals window	View → Locals	See every variable in scope
Watch	Right-click a variable → Add Watch	Stop when a value changes or meets a condition
Debug.Print	In the code	A log of what happened, without dialogs
Stop	In the code	A breakpoint that travels with the file

The routine when a macro misbehaves: set a breakpoint before the suspect line, run, then step with **F8** while watching Locals. Nine times out of ten the problem is a variable that isn't what you assumed: the wrong sheet, an empty string, a number stored as text.

Error handling

```
Sub RunConsolidation()  
    Dim errorLog As String  
  
    On Error GoTo Failed                ' from here, any error jumps to the label  
  
    Application.ScreenUpdating = False  
    ConsolidateBranchFiles  
    CleanMaster  
    BuildSummaryPivot  
  
Cleanup:  
    Application.ScreenUpdating = True  
    Application.DisplayAlerts = True  
    Application.Calculation = xlCalculationAutomatic  
    Exit Sub                          ' stop before falling into the handler  
  
Failed:  
    errorLog = "Error " & Err.Number & ": " & Err.Description & " (in " & Err.Source & ")"  
    Debug.Print errorLog  
    MsgBox errorLog & vbCrLf & vbCrLf & "Nothing was emailed.", vbCritical, "Consolidation failed"  
    Resume Cleanup                    ' always restore Excel's settings  
End Sub
```

- **On Error GoTo Label** sends control to a handler. `Err.Number` and `Err.Description` say what happened.
- **Resume Cleanup** continues at a label, which is how you guarantee `ScreenUpdating` gets switched back on. A macro that fails with screen updating off leaves Excel looking frozen, and that's how users learn to fear macros.
- **On Error Resume Next** ignores errors and continues. It has exactly two legitimate uses: deleting something that may not exist, and testing whether an object exists. Turn it off immediately afterwards with `On Error GoTo 0`.
- **Fail loudly, not silently.** If the consolidation read eleven files instead of twelve, the macro should say so and refuse to email.

A simple log file is worth the ten lines:

```
Sub LogLine(message As String)  
    Dim f As Integer  
    f = FreeFile  
    Open ThisWorkbook.Path & Application.PathSeparator & "macro_log.txt" For Append As #f  
    Print #f, Format(Now, "yyyy-mm-dd HH:mm:ss") & " " & message  
    Close #f  
End Sub
```

19.10 Making macros fast

A macro that takes four minutes gets switched off. Three changes usually take it under ten seconds.

1. Stop Excel redrawing and recalculating

```
Sub FastWrapper()  
    Dim calcMode As XlCalculation  
    calcMode = Application.Calculation  
  
    Application.ScreenUpdating = False  
    Application.Calculation = xlCalculationManual  
    Application.EnableEvents = False  
    Application.DisplayStatusBar = False  
  
    ' ... the work ...  
  
    Application.DisplayStatusBar = True  
    Application.EnableEvents = True  
    Application.Calculation = calcMode  
    Application.ScreenUpdating = True  
End Sub
```

Always restore them (see the error handler above). `EnableEvents = False` also stops your own `Worksheet_Change` code from firing on every write.

2. Work in arrays, not cell by cell

Every read or write to a cell crosses the boundary between VBA and Excel, and that crossing is the expensive part. Read once into an array, work in memory, write once.

```
Sub CleanStatusFast()  
    Dim ws As Worksheet, lastRow As Long, r As Long  
    Dim values As Variant  
    Dim statusMap As Object  
  
    Set ws = ThisWorkbook.Worksheets("Master")  
    lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row  
  
    Set statusMap = CreateObject("Scripting.Dictionary")  
    statusMap("delivered") = "Delivered": statusMap("dlvd") = "Delivered"  
    statusMap("cancelled") = "Cancelled": statusMap("canceled") = "Cancelled"  
    statusMap("cxl") = "Cancelled": statusMap("shipped") = "Shipped"  
    statusMap("pending") = "Pending"  
  
    values = ws.Range(ws.Cells(2, 9), ws.Cells(lastRow, 9)).Value2 ' one read  
    For r = 1 To UBound(values, 1)  
        Dim raw As String  
        raw = LCase(Trim(CStr(values(r, 1))))  
        If statusMap.Exists(raw) Then values(r, 1) = statusMap(raw)  
    Next r  
    ws.Range(ws.Cells(2, 9), ws.Cells(lastRow, 9)).Value2 = values ' one write  
End Sub
```

On 25,832 rows, the cell-by-cell version in section 19.6 takes several seconds; this one is effectively instant. `.Value2` is slightly faster than `.Value` and doesn't convert dates and currency, which is usually what you want when moving data around.

3. Don't do what Excel can do in one line

Instead of looping to...	Use
Copy a range	<code>src.Copy dest</code>
Find a value	<code>Range.Find</code>
Filter rows	<code>AutoFilter</code> + <code>SpecialCells(xlCellTypeVisible)</code>
Remove duplicates	<code>Range.RemoveDuplicates</code>
Sum by a key	A pivot table, or <code>Application.WorksheetFunction.SumIfs</code>
Sort	<code>Range.Sort</code>
Delete matching rows	<code>AutoFilter</code> , then delete the visible rows in one go

And the honest version of all of this: **if the job is "combine these files and reshape them", Power Query does it with no code at all** (Chapter 11), refreshes on demand, and doesn't need a macro-enabled file. Reach for VBA when you need the things Power Query can't do: formatting, PDFs, email, and driving Excel itself.

19.11 Office Scripts: Excel on the web

VBA doesn't run in Excel on the web. **Office Scripts** do: TypeScript (JavaScript with types) recorded or written in the browser, stored with your account, and runnable from **Automate → All Scripts** or from a **Power Automate** flow.

	VBA	Office Scripts
Runs in	Excel for Windows/Mac (desktop)	Excel on the web (and desktop, by running the cloud script)
Language	VBA	TypeScript
Stored in	The workbook, or PERSONAL.XLSB	Your OneDrive / SharePoint, with the workbook
Can open other files	Yes	No: one workbook per run (Power Automate passes data between steps)
Can send email	Yes, via Outlook on the machine	Not directly: Power Automate does it
Runs unattended	Only with the machine on and Excel open	Yes, through Power Automate in the cloud
Availability	Any Excel with macros enabled	Microsoft 365 business plans (not personal accounts); check your plan

The same header formatting as section 19.2:

```
function main(workbook: ExcelScript.Workbook) {
  const sheet = workbook.getWorksheet("Master");
  const used = sheet.getUsedRange();
  const lastRow = used.getRowCount();
  const lastCol = used.getColumnCount();

  const header = sheet.getRangeByIndexes(0, 0, 1, lastCol);
  header.getFormat().getFont().setBold(true);
  header.getFormat().getFont().setColor("#FFFFFF");
  header.getFormat().getFill().setColor("#0F5C8C");
  sheet.getRange("A2").getFormat().setRowHeight(18);
  used.getFormat().autofitColumns();

  console.log(`formatted ${lastRow} rows, ${lastCol} columns`);
}
```

Reading and writing in bulk, which matters even more here than in VBA because every call crosses the network:

```
function main(workbook: ExcelScript.Workbook) {
  const sheet = workbook.getWorksheet("Master");
  const range = sheet.getUsedRange();
  const values = range.getValues(); // one read: a 2-D array

  const map: { [key: string]: string } = {
    "delivered": "Delivered", "dlvd": "Delivered", "cancelled": "Cancelled",
    "canceled": "Cancelled", "cxl": "Cancelled", "shipped": "Shipped", "pending": "Pending"
  };

  let unmapped = 0;
  for (let r = 1; r < values.length; r++) { // row 0 is the header
    const raw = String(values[r][8]).trim().toLowerCase(); // column I
    if (map[raw]) { values[r][8] = map[raw]; } else { unmapped++; }
  }

  range.setValues(values); // one write
  console.log(`${values.length - 1} rows, ${unmapped} unmapped statuses`);
}
```

Power Automate is where Office Scripts become an automation rather than a button: a flow can run on a schedule or a trigger (a new file in SharePoint, a form response), run the script, and then email the result, post to Teams, or write to another system. That combination replaces most of what VBA did, for organizations that live in SharePoint and Teams. It's also the honest answer when IT won't allow macros.

19.12 Google Sheets: macros and Apps Script

Google Sheets has a recorder (**Extensions → Macros → Record macro**) that writes **Apps Script**, which is JavaScript running on Google's servers. **Extensions → Apps Script** opens the editor.

```
function formatMaster() {
  const sheet = SpreadsheetApp.getActive().getSheetByName('Master');
  const lastRow = sheet.getLastRow();
  const lastCol = sheet.getLastColumn();

  sheet.getRange(1, 1, 1, lastCol)
    .setFontWeight('bold')
    .setFontColor('#ffffff')
    .setBackground('#0f5c8c');
  sheet.setFrozenRows(1);
  sheet.autoResizeColumns(1, lastCol);

  Logger.log(`formatted %s rows and %s columns`, lastRow, lastCol);
}
```

Read and write in batches

Apps Script has quotas and a six-minute limit per execution, and every call to the spreadsheet is a round trip. The single most important habit is the same as VBA's arrays:

```
function cleanStatuses() {
  const sheet = SpreadsheetApp.getActive().getSheetByName('Master');
  const range = sheet.getDataRange();
  const values = range.getValues(); // one read

  const map = {
    'delivered': 'Delivered', 'dlvd': 'Delivered', 'cancelled': 'Cancelled',
    'canceled': 'Cancelled', 'cxl': 'Cancelled', 'shipped': 'Shipped', 'pending': 'Pending'
  };

  let unmapped = 0;
  for (let r = 1; r < values.length; r++) {
    const raw = String(values[r][8]).trim().toLowerCase();
    if (map[raw]) values[r][8] = map[raw];
    else unmapped++;
  }

  range.setValues(values); // one write
  Logger.log('%s rows cleaned, %s unmapped', values.length - 1, unmapped);
}
```

A loop that calls `getRange().setValue()` per cell on 25,000 rows will hit the six-minute limit. The batch version finishes in seconds.

A custom menu, so colleagues can run it

```
function onOpen() {
  SpreadsheetApp.getUi()
    .createMenu('Riverstone')
    .addItem('Clean statuses', 'cleanStatuses')
    .addItem('Send daily summary', 'sendDailySummary')
    .addToUi();
}
```

`onOpen` is a **simple trigger**: Google runs it when the sheet opens, and the menu appears next to Help.

Custom functions

```
/**
 * Riverstone's loyalty rebate percentage.
 * @param {number} annualValue The customer's annual net revenue.
 * @return {number} 0, 1 or 2.
 * @customfunction
 */
function REBATEPCT(annualValue) {
  if (annualValue >= 400000) return 2;
  if (annualValue >= 250000) return 1;
  return 0;
}
```

The `@customfunction` tag makes it available as `=REBATEPCT(B2)`. Custom functions can't call services that need authorization (no email, no external API with credentials), and they're cached, so they're for calculations only.

19.13 Triggers, email, and APIs in Apps Script

Triggers

Trigger	Fires when	Set up
<code>onOpen</code> , <code>onEdit</code>	The sheet is opened or edited	Simple triggers: name the function and Google runs it
Time-driven	Every hour, every day at 9 a.m., every Monday	Triggers page in the editor, or <code>ScriptApp.newTrigger(...)</code>
On form submit	A linked Google Form is submitted	Installable trigger on the sheet or the form
On change	Structure changes (rows added, sheet renamed)	Installable trigger

Installable triggers run as **you**, with your permissions, even when you're asleep. That's the power and the risk: a trigger that emails customers is sending mail from your account.

An acknowledgement email on form submit

```
function onFormSubmit(e) {
  const row = e.namedValues;
  const email = row['Email address'][0];
  const company = row['Company'][0];
  const product = row['Product'][0];
  const quantity = row['Quantity'][0];

  const html = `
    <p>Dear ${company}</p>
    <p>Thank you for your enquiry. We have logged it and a sales executive will reply within one working day.</p>
    <table style="border-collapse:collapse;font-family:Arial;font-size:13px">
      <tr><td style="padding:4px 12px;color:#5b6475">Product</td><td style="padding:4px 12px"><b>${product}</b></td></tr>
      <tr><td style="padding:4px 12px;color:#5b6475">Quantity</td><td style="padding:4px 12px"><b>${quantity}</b></td></tr>
    </table>
    <p style="color:#5b6475;font-size:12px">Riverstone Supplies · this is an automatic acknowledgement.</p>`;

  MailApp.sendEmail({
    to: email,
    subject: `Riverstone: we received your enquiry (${product})`,
    htmlBody: html,
    name: 'Riverstone Supplies'
  });

  SpreadsheetApp.getActive().getSheetByName('Enquiries')
    .appendRow([new Date(), email, company, product, quantity, 'acknowledged']);
}
```

A 9 a.m. summary in the email body

```
function sendDailySummary() {
  const sheet = SpreadsheetApp.getActive().getSheetByName('Enquiries');
  const values = sheet.getDataRange().getValues().slice(1);

  const yesterday = new Date(); yesterday.setDate(yesterday.getDate() - 1);
  const isYesterday = (d) => new Date(d).toDateString() === yesterday.toDateString();
  const rows = values.filter(r => isYesterday(r[0]));

  const byProduct = {};
  rows.forEach(r => { byProduct[r[3]] = (byProduct[r[3]] || 0) + Number(r[4]); });

  const tableRows = Object.keys(byProduct).sort()
    .map(p => `<tr><td style="padding:4px 12px">${p}</td>
      <td style="padding:4px 12px;text-align:right">${byProduct[p].toLocaleString('en-IN')}</td></tr>`)
    .join('');

  const html = `
    <h3 style="font-family:Arial">Enquiries yesterday: ${rows.length}</h3>
    ${rows.length === 0 ? '<p>No enquiries were received.</p>' :
      `<table style="border-collapse:collapse;font-family:Arial;font-size:13px">
        <tr><th style="text-align:left;padding:4px 12px;border-bottom:1px solid #c9d3df">Product</th>
          <th style="text-align:right;padding:4px 12px;border-bottom:1px solid #c9d3df">Units</th></tr>
          ${tableRows}
        </table>`}
    <p style="color:#5b6475;font-size:12px">Sent automatically at ${new Date().toLocaleString('en-IN')}</p>`;

  MailApp.sendEmail({ to: 'sales.team@riverstone.example', subject: 'Riverstone enquiries – daily summary',
    htmlBody: html, name: 'Riverstone reporting' });
}
```

Note the "no data today" branch: an automation that says "*no enquiries were received*" is trustworthy; one that sends an empty table looks broken, and one that sends nothing at all leaves people wondering (Chapter 20).

Calling an API

```
function fetchOpenEnquiries() {
  const token = PropertiesService.getScriptProperties().getProperty('CRM_TOKEN'); // not in the code
  const response = UrlFetchApp.fetch('https://crm.example.com/api/v1/enquiries?status=open', {
    method: 'get',
    headers: { Authorization: 'Bearer ' + token },
    muteHttpExceptions: true
  });

  if (response.getResponseCode() !== 200) {
    throw new Error('CRM returned ' + response.getResponseCode() + ': ' + response.getContentText().slice(0, 200));
  }

  const data = JSON.parse(response.getContentText());
  const rows = data.results.map(r => [r.id, r.company, r.product, r.quantity, r.created_at]);
  const sheet = SpreadsheetApp.getActive().getSheetByName('CRM');
  sheet.getRange(2, 1, rows.length, rows[0].length).setValues(rows);
  Logger.log('%s enquiries written', rows.length);
}
```

- **PropertiesService** stores the token outside the code, which matters because Apps Script files are visible to anyone who can edit the sheet.
- **muteHttpExceptions: true** lets you read the status code and message instead of the script dying with a generic error.
- **Quotas** apply per account per day: **MailApp** is limited (100 recipients a day on consumer accounts, 1,500 on Workspace at the time of writing), **UrlFetchApp** has a daily call limit, and each execution must finish within six minutes. Check the current quota page before promising a volume.

19.14 The same job in three languages

Consolidating and cleaning, as it appears in each environment:

Task	VBA	Office Scripts	Apps Script
Get the sheet	Set ws = ThisWorkbook.Worksheets("Master")	const sheet = workbook.getWorksheet("Master")	const sheet = SpreadsheetApp.getActive().getSheetByName('Master')
Last row	ws.Cells(ws.Rows.Count, "A").End(xlUp).Row	sheet.getUsedRange().getRowCount()	sheet.getLastRow()
Read a block	v = rng.Value2	const v = range.getValues()	const v = range.getValues()
Write a block	rng.Value2 = v	range.setValues(v)	range.setValues(v)
Loop	For r = 1 To UBound(v, 1)	for (let r = 0; r < v.length; r++)	for (let r = 0; r < v.length; r++)
Lookup table	Scripting.Dictionary	object literal {} or Map	object literal {} or Map
Bold the header	.Font.Bold = True	.getFormat().getFont().setBold(true)	.setFontWeight('bold')
Log	Debug.Print	console.log	Logger.log
Open other files	Workbooks.Open	not available	SpreadsheetApp.openById
Email	Outlook object	via Power Automate	MailApp / GmailApp
Schedule	Windows Task Scheduler	Power Automate	Time-driven trigger

The shapes are identical: get a range, read once, loop in memory, write once, log what happened. Learn the pattern and the syntax is a lookup.

19.15 Living with macros responsibly

A macro becomes a liability the day its author leaves. These habits prevent that.

Write it so someone else can read it

- `Option Explicit` at the top; meaningful names; one procedure, one job.
- A comment block at the top of the module: what it does, what it expects (folder, sheet names, columns), what it produces, who owns it, when it was last changed.
- No magic numbers: `Const` for paths, sheet names, column positions, and thresholds. Better still, find columns by header name rather than by position, so an inserted column doesn't silently break the totals.

Never hard-code secrets or personal paths

- `C:\Users\meera\Desktop\` works on exactly one machine. Use `ThisWorkbook.Path`, a folder picker, or a named cell in a settings sheet.
- Passwords and tokens don't belong in a module. Apps Script has `PropertiesService`; VBA has no good answer, which is itself a reason to move that job to Python (Chapter 18, section 18.13) or Power Automate.
- A settings sheet (`Config`) with named cells for folder, recipients, and month is the simplest way to let the business change what it should change without touching code.

Version control

VBA lives inside a binary file, so Git can't diff it. Two workable answers: export the modules (`.bas` , `.cls` , `.frm`) to a folder and commit those files (right-click the module → **Export File**), or keep dated copies of the workbook with a changelog sheet. Apps Script has built-in version history and a Git-friendly command-line tool (`clasp`). Office Scripts keep versions in OneDrive. Chapter 26 covers Git properly.

The single-owner risk

If one person understands the macro, the business has a dependency, not an asset. Fix it with: a one-page document (what it does, how to run it, what to do when it fails), a colleague who has run it once, and code readable enough that a competent analyst could pick it up. Ask yourself the honest question: *if I'm on leave and it fails on the 2nd, what happens?*

When to stop using macros

Sign	Move to
The macro is mostly combining and reshaping files	Power Query (Chapter 11)
It needs data from a database or an API	Python (Chapter 18)
It must run when nobody is logged in	Python plus a scheduler, or Power Automate (Chapter 20)
Several people need the output, interactively	Power BI (Chapter 16)
It's over about 500 lines, or nobody dares change it	Rewrite, in whatever tool fits, with tests
It handles personal or financial data with emails	A reviewed process, not a macro on one laptop

None of that makes VBA a waste of time. It makes it what it is: the fastest way to automate the workbook in front of you, and a skill that pays for itself the first month you use it.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Saving a macro workbook as <code>.xlsx</code>	The code is gone next time	Save as <code>.xlsm</code> ; check the title bar
Telling users to enable all macros	A security incident waiting to happen	Trusted locations, signed macros
Keeping recorded code as-is	Breaks when the data changes size	Rewrite with variables and the last-row pattern
Select and Activate everywhere	Slow; breaks when focus changes	Address objects directly
No <code>Option Explicit</code>	A typo silently creates an empty variable	Turn on Require Variable Declaration
Integer for row counts	Overflow above 32,767 rows	Long
Hard-coded paths and sheet names	Runs on one laptop only	<code>Const</code> , a settings sheet, or a folder picker
Hard-coded column positions	An inserted column breaks the totals silently	Find columns by header text
Cell-by-cell loops	Minutes instead of seconds	Read into an array, work, write back
<code>ScreenUpdating</code> left off after an error	Excel looks frozen; users panic	Error handler with <code>Resume</code> <code>CleanUp</code>

Mistake	Symptom	Fix
On Error Resume Next across whole procedures	Errors hidden; wrong results shipped	Handle specific cases; On Error GoTo 0
Emailing with .Send while developing	The wrong draft reaches the sales team	.Display until you trust it
No source-file column when consolidating	You can't trace a wrong number or spot a double-count	Add source_file
Copying the header from every file	Header rows scattered through the data	Copy the header once, data from row 2
Cell-by-cell calls in Apps Script	Six-minute timeout	getValues() / setValues() once
Tokens in the script file	Anyone who can edit the sheet can read them	PropertiesService, or move the job
One person understands it	The business depends on a person, not a process	Document, share, simplify
A macro that should be Power Query	Hundreds of lines doing a refreshable job	Rebuild it in Power Query

In the real world: the macro that ran for nine years

When Meera joined Riverstone, the Q4 branch consolidation ran on a workbook called `MASTER_FINAL_v7_USE_THIS.xlsm`, written in 2017 by an analyst who had left in 2020. Every quarter, someone opened it, clicked a gray button labeled "RUN", waited about four minutes while the screen flickered, and emailed the PDF it produced.

It mostly worked. Three things about it were quietly expensive.

It counted whatever files it found. The macro looped `Dir("*.xls*")` over a shared folder. In the quarter Meera arrived, that folder had thirteen files: the twelve branch extracts and one called `Riverstone_Kolkata_2025-11 (2).xlsx`, a copy someone had saved when the network dropped. The total was ₹18.8 million too high, and nobody noticed, because the macro reported nothing but "Done!".

It stopped at row 5,000. The loop ran `For r = 2 To 5000`, a number the original author had picked as "plenty". October 2025's Mumbai file had 3,297 rows, so it was fine, but the consolidated sheet was silently truncated the first time a branch grew past the limit.

Nobody could read it. 740 lines, no `Option Explicit`, variables called `x`, `x2`, and `temp3`, and a hard-coded path to a laptop that no longer existed, worked around by a mapped drive that IT wanted to remove.

She rewrote it in two afternoons, not as a heroic act but as a set of ordinary decisions:

- The folder comes from a `Config` sheet, and the macro **lists the files it read** in the summary, with row counts per file.
- Rows come from `Cells(Rows.Count, "A").End(xlUp).Row`, not from a guess.
- Each row carries its `source_file`, so a suspicious number can be traced in ten seconds.
- Three checks run before anything is emailed: twelve files found, no duplicate file names after normalizing, and the consolidated total within 25% of the same quarter last year. If a check fails, the macro writes to a log, shows what failed, and refuses to send.
- The reading loop became an array read, and the macro went from four minutes to nine seconds.
- A one-page note lives in the workbook's first sheet: what it does, how to run it, what each check means, and who to call.

The first run after the rewrite failed the duplicate check and named the file. That, in Meera's report to Anita Rao, was the entire business case: *"The old macro would have added ₹18.8 million of a branch's November sales twice, as it probably has before. The new one won't send a report it can't stand behind."*

What made the difference:

- **It reports what it did**, not merely that it finished.
- **It checks before it sends**, and fails loudly.
- **It has no secrets**: no hidden paths, no assumptions about size, no cleverness.
- **Someone else can run it**, which is the only real test of an automation.

Tools

- **Excel for Windows** (Microsoft 365 or 2016+) with the **Developer** tab enabled, for VBA, Outlook email, and PDF export. Excel for Mac runs VBA but not the Outlook automation.

- **Excel on the web** with a Microsoft 365 **business** plan, for Office Scripts and Power Automate. Personal Microsoft accounts don't have Office Scripts.
- **A Google account** for Sheets, Apps Script, Forms, and triggers. Workspace accounts have much higher email quotas than consumer ones.
- **Companion files (companion/ch19/):**
 - `build_ch19_files.py` builds everything below.
 - `branch_files/` : twelve workbooks (four branch sales offices × three months of Q4 2025), plus `_notes.txt` .
 - `expected_results.md` : the row count and net revenue of every file, and the totals a correct consolidation must produce.
 - `enquiries_sample.csv` : sample Google Form responses for the Apps Script project.
 - `vba/` : the chapter's VBA modules as `.bas` files, ready to import (**File → Import File** in the editor).
 - `office_scripts/` and `apps_script/` : the TypeScript and JavaScript versions.

Note on this chapter's code. Everything in Chapters 17 and 18 was run and its output checked automatically. VBA, Office Scripts, and Apps Script can't be run in that environment, so this chapter's code was written and reviewed by hand, and the **data** it processes was generated and checked with Python: `expected_results.md` is the yardstick. Run the macros in Excel and compare.

The project: two automations, end to end

Part 1: the Excel consolidation (VBA)

Goal: one button that turns twelve branch files into a summary, a PDF, and an email.

1. **Set up** a new `.xlsm` workbook with sheets `Config` , `Master` , and `Summary` . Put the folder path, the recipient list, and the quarter in named cells on `Config` .
2. **Consolidate** every `.xlsx` in the folder into `Master` , with a `source_file` column, ignoring `_notes.txt` .
3. **Clean:** standardize `status` through a dictionary, keep `customer_code` as four digits, and flag anything unmapped.
4. **Check, before anything else happens:** twelve files read; no duplicate file names; row count matches the sum of the per-file counts; net revenue within a sensible range. Write the results to a log sheet.
5. **Summarize:** a pivot of net revenue by branch (from `source_file`) and month, plus the headline numbers.
6. **Export** the summary as a PDF named with the quarter and today's date.
7. **Email** it with an HTML body containing the headline numbers, using `.Display` until you're sure, then `.Send` .
8. **Wrap** the whole thing in one `RunConsolidation` procedure with error handling that restores Excel's settings and refuses to email if a check failed.

What good looks like: 25,832 data rows, 24,738 non-cancelled, net revenue ₹423,872,808.00; by branch, Mumbai HO ₹151,058,519.25 · Bengaluru ₹117,726,527.00 · Delhi ₹102,966,828.00 · Kolkata ₹52,120,933.75; by month, October ₹180,620,103.00 · November ₹155,985,902.00 · December ₹87,266,803.75. The whole run should take under 15 seconds.

Part 2: the Google Sheets workflow (Apps Script)

Goal: an enquiry form that answers itself, and a daily summary that sends itself.

1. **Create a Google Form** with the fields in `enquiries_sample.csv` (email, company, city, product, quantity, needed by, notes) and link it to a sheet.
2. **On submit:** send the customer an HTML acknowledgement, and append a row to an `Enquiries` sheet with a status of `acknowledged` .
3. **Validate:** if the quantity isn't a number or the email looks wrong, write the row to an `Exceptions` sheet and notify the sales team instead of the customer.
4. **At 9 a.m. every weekday:** email the sales team yesterday's enquiries as an HTML table in the body, grouped by product, with a "no enquiries were received" branch.
5. **Add a menu** so someone can run the summary by hand.
6. **Protect it:** no addresses or tokens in the code, a log sheet of what was sent and when, and a note in the sheet explaining who owns it.

Stretch goals

- Rebuild Part 1's consolidation in Power Query and compare: lines of code, time to run, and what each can't do.
- Rewrite Part 1's cleaning step as an Office Script and run it from Power Automate on a schedule.
- Add a quarter-on-quarter comparison to the PDF, using last quarter's saved summary.

Timed challenge: forty-five minutes in the editor

Use `companion/ch19/branch_files/` and a blank `.xlsm`. Answers at the end of the chapter.

- **Level 1:** Write a macro that counts the `.xlsx` files in the folder and prints the count and each file name to the Immediate window. How many files, and how many does `*.xls*` match?
- **Level 2:** Open each file and print its row count (excluding the header). What's the total?
- **Level 3:** Consolidate all twelve into one sheet with a `source_file` column. How many data rows?
- **Level 4:** Total `net_revenue` for rows whose status is not `Cancelled`.
- **Level 5:** Produce the total per branch, using the file name to derive the branch.
- **Level 6:** Produce the total per month, the same way.
- **Level 7:** Rewrite your row loop to read the data into an array and time both versions with `Timer`.
- **Bonus:** Add a check that fails if fewer than twelve files were read, and make the macro refuse to continue.

You've got it when...

- ☐ You can record a macro, read what it wrote, and rewrite it without `Select`.
- ☐ You know which file types hold macros, and how to handle macro security without disabling it.
- ☐ You declare variables with sensible types, use `Option Explicit`, and can write `If`, `Select Case`, and all three loops from memory.
- ☐ You use the last-row pattern instead of fixed ranges, and address objects instead of activating them.
- ☐ You can consolidate a folder of files, with a source column and a per-file row count.
- ☐ You can clean a column through a dictionary and flag what isn't in the map.
- ☐ You can build a pivot, export a PDF, and send an Outlook email with an HTML body.
- ☐ You debug with breakpoints, the Immediate window, and Locals, and handle errors with a handler that restores Excel's settings.
- ☐ You make a slow macro fast with arrays and `ScreenUpdating`.
- ☐ You can do the same job in Office Scripts and Apps Script, and say when each is the right home.
- ☐ Your automations carry a settings sheet, a log, checks that can stop them, and a note for whoever inherits them.

Recap

- **Spreadsheet automation is worth doing** when the data and the audience already live in workbooks; Power Query, Power BI, and Python are better for everything else.
- **Macros live in** `.xlsm`, run under macro security, and belong in a trusted location rather than behind "enable everything".
- **Record to learn, then rewrite:** recordings select, are literal, and depend on what's active.
- **VBA basics:** `Option Explicit`, `Dim` with real types (`Long`, not `Integer`), `If / Select Case`, `For / For Each / Do`, arrays, `Scripting.Dictionary`, `Sub` and `Function` with arguments.
- **The object model** is `Application` → `Workbook` → `Worksheet` → `Range`. The last-row pattern, `Cells(row, col)`, and `With` cover most of what you write. Avoid `Select` and `ActiveSheet`.
- **The toolkit:** consolidate a folder with `Dir`, add a `source_file` column, clean through a dictionary, build a pivot, export a PDF, email through Outlook with `.Display` first.
- **Debug** with breakpoints, `Debug.Print`, the Immediate window; **handle errors** with `On Error GoTo` and a `Resume Cleanup` that restores settings.

- **Make it fast** with `ScreenUpdating = False`, manual calculation, and arrays instead of cell-by-cell access: 25,832 rows go from seconds to instant.
- **Office Scripts** bring the same ideas to Excel on the web in TypeScript, and run unattended through Power Automate; **Apps Script** does it for Sheets in JavaScript, with triggers, `MailApp`, `UrlFetchApp`, and quotas to respect.
- **Responsibility**: settings not hard-coded, secrets not in code, checks before sending, a log, version control for exported modules, and a plan for the day you're not there.

Practice exercises

Use `companion/ch19/branch_files/` and `expected_results.md`.

Warm-up

1. Which file extensions can hold macros, and what happens if you save a macro workbook as `.xlsx`?
2. What are the three problems with almost every recorded macro? Rewrite this recording properly: `Sheets("Master").Select / Range("A1").Select / ActiveCell.FormulaR1C1 = "Total"`.
3. Why is `Option Explicit` worth turning on, and what exactly goes wrong without it?
4. Why is `Dim rowCount As Integer` a bug waiting to happen in a data macro?
5. Write the one line that finds the last used row in column A of a worksheet `ws`, and explain each part.
6. When would you use `For Each` rather than `For ... Next`? Give a Riverstone example of each.

Core

7. Record a macro that bolds row 1, freezes it, and autofits the columns of a branch file. Then rewrite it as a `Sub` that takes a `Worksheet` argument.
8. Write a macro that prints the name and row count of every `.xlsx` in the branch folder. How many files are there, and what is the total row count?
9. Consolidate the twelve files into a `Master` sheet with a `source_file` column. How many data rows, and does it match `expected_results.md`?
10. Add the headline calculations: non-cancelled rows and their net revenue. What are they?
11. Produce net revenue per branch, deriving the branch from the file name. Which branch is largest, and by how much over the smallest?
12. Produce net revenue per month, and state October's share of the quarter.
13. Standardize the `status` column with a `Scripting.Dictionary`, and count how many values weren't in the map.
14. Write a macro that formats the `Master` sheet: header style, number format on `net_revenue`, frozen header, autofit, and a filter.
15. Build a pivot table of net revenue by `source_file` and `status` in VBA.
16. Export the summary sheet to PDF with a file name that includes today's date, in landscape, fitted to one page wide.
17. Write the email procedure with `.Display`, including an HTML body with the headline numbers. What would you change before switching to `.Send`?
18. Write `REBATEPCT` as a UDF and check it on 180,000, 250,000, 399,999, and 400,000.
19. Add error handling to your main procedure that restores `ScreenUpdating` and `Calculation`, logs the error, and prevents the email.
20. Time your cleaning loop with `Timer`, then rewrite it with an array read and write. What's the difference on 25,832 rows?
21. Write the Apps Script version of the cleaning step for a Google Sheets copy of `Master`, using one `getValues()` and one `setValues()`.
22. Write an Apps Script `onOpen` menu with two items, and a time-driven trigger that runs a summary at 9 a.m.

Stretch

23. Rewrite the consolidation so it finds columns by header name rather than position, and still works if a branch adds a column.
24. Add three checks that must pass before the email is sent, including one that compares the total with a previous run stored on a `History` sheet.
25. Rebuild the consolidation in Power Query and write ten lines comparing it with the macro: effort, speed, maintainability, and what each can't do.

26. Export your modules as `.bas` files and put them under Git. What does the diff of a small change look like, and why is that better than versioning the `.xls`?

Think about it

27. Your predecessor's macro sends a report to 40 people every Monday, and you can't read the code. What do you do first, second, and third?
28. When is it right to keep a working macro rather than rewrite it in Python, even though Python would be "better"?

Key terms

macro · macro recorder · `.xls` · `.xlsb` · macro security · trusted location · Mark of the Web · Personal Macro Workbook · form control · VBA · Visual Basic Editor · module · `Option Explicit` · `Dim` · `Set` · `Long` versus `Integer` · `Select Case` · `For Each` · `Do While` · `array` · `Scripting.Dictionary` · `Sub` · `Function` · `ByVal` / `ByRef` · object model · `Application` · `Workbook` · `Worksheet` · `Range` · `Cells` · `End(xlUp)` · `CurrentRegion` · `With` · `Value2` · `Dir` · `FileSystemObject` · `PivotCache` · `ExportAsFixedFormat` · late binding · `CreateObject` · `HTMLBody` · `UDF` · `UserForm` · `InputBox` · `FileDialog` · breakpoint · Immediate window · `Debug.Print` · `On Error GoTo` · `Resume` · `Err` · `ScreenUpdating` · `Calculation` · `EnableEvents` · Office Scripts · TypeScript · Power Automate · Apps Script · `SpreadsheetApp` · `getValues` / `setValues` · simple and installable triggers · `MailApp` · `GmailApp` · `UrlFetchApp` · `PropertiesService` · `quota` · `clasp`

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 20, Automating Reports & Delivering Insights:** scheduling, HTML email at scale, alerts, failure handling, and choosing between VBA, Apps Script, Python, and low-code flows.
- **Chapter 11** remains the first answer for refreshable data shaping; **Chapter 16** for shared, interactive reporting.
- **Chapter 18, Python for Analysts:** the same automations when the data comes from a database or an API, or must run unattended.
- **Chapter 26, Git:** versioning exported modules and scripts.
- **Chapter 46:** orchestration, when an automation grows into a pipeline with dependencies and retries.
- **Interview preparation:** the Excel, Google Sheets, VBA & BI Question Bank (Chapter 70) covers the object model, the last-row pattern, and "how would you make this macro faster?".

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. `.xls`, `.xlsb`, and `.xlsm` (and the legacy `.xls`). Saving as `.xlsx` strips the code, with a single warning: the macro is gone when the file reopens.
2. They select things, they're literal about ranges and sheets, and they depend on what's active. Rewritten:
`ThisWorkbook.Worksheets("Master").Range("A1").Value = "Total"`.
3. It forces `Dim` for every variable, so a misspelling is a compile error instead of a new, empty variable. Without it, `netRevene = total` silently creates a second variable and your reported total stays zero.
4. `Integer` overflows above 32,767. Excel has 1,048,576 rows, and the consolidated sheet here has 25,833 including the header, so a row counter will eventually raise "Overflow". Use `Long`.
5. `lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row` — start at the bottom of the sheet in column A (`ws.Rows.Count` is the last row number), press `Ctrl+Up` (`End(xlUp)`), and take that cell's row number.
6. `For Each` when looping a collection whose size you don't care about: every worksheet in the workbook, every file in a folder collection. `For ... Next` when you need the index: rows 2 to `lastRow`, or counting backwards while deleting rows.
7. See section 19.2. The rewritten version takes `ws As Worksheet`, uses `With ws`, and never touches `ActiveSheet`.

8. 12 files. Total data rows **25,832**: Bengaluru 2,631 + 2,481 + 2,073; Delhi 2,211 + 2,220 + 1,843; Kolkata 1,117 + 1,099 + 994; Mumbai HO 3,297 + 3,167 + 2,699.
9. 25,832 data rows in **Master** (25,833 including the header). If you get 25,844, you copied a header row from each file; if you get fewer, a file was skipped or a last-row calculation is wrong.
10. **24,738** non-cancelled rows, net revenue **₹423,872,808.00**. (All rows, including cancelled: ₹442,577,334.00.)
11. Mumbai HO **₹151,058,519.25**, Bengaluru ₹117,726,527.00, Delhi ₹102,966,828.00, Kolkata **₹52,120,933.75**. Mumbai HO is ₹98,937,585.50 above Kolkata, close to three times its size.
12. October ₹180,620,103.00 · November ₹155,985,902.00 · December ₹87,266,803.75. October is **42.6%** of the quarter.
13. On this clean data every status maps, so the count of unmapped values is **0**. That's the point of the check: it's how you find the day a branch starts writing something new.
14. Any correct formatting macro. Test it by adding a column to one branch file and rerunning: if your code uses fixed column letters, it now formats the wrong column.
15. See section 19.6. The pivot's grand total must equal ₹442,577,334.00 across all statuses, or ₹423,872,808.00 if you filter Cancelled out.
16. `ws.PageSetup.Orientation = xlLandscape, ws.PageSetup.Zoom = False, ws.PageSetup.FitToPagesWide = 1, ws.PageSetup.FitToPagesTall = False`, then `ExportAsFixedFormat` with a name built from `Format(Date, "yyyy-mm-dd")`.
17. Before switching to `.Send`: check the recipient list comes from the `Config` sheet and not from a hard-coded string, that the checks passed, that the PDF exists and isn't zero bytes, and that the subject and body contain the right period. Send it to yourself first.
18. 180,000 → 0; 250,000 → 1; 399,999 → 1; 400,000 → 2.
19. See section 19.9. The key line is `Resume CleanUp`, so `ScreenUpdating`, `Calculation`, and `DisplayAlerts` are restored whatever happened, and the email procedure is called only after the checks.
20. The cell-by-cell version takes a few seconds on 25,832 rows (longer with screen updating on); the array version is effectively instant. Report both with `Timer` before and after: `Debug.Print Format(Timer - t0, "0.00") & " seconds"`.
21. See section 19.12: `getDataRange().getValues()`, loop the array, map the status, `setValues()` once. A per-cell version on 25,832 rows will hit the six-minute limit.
22. `onOpen` builds the menu with `SpreadsheetApp.getUi().createMenu(...)`. The 9 a.m. job is an installable time-driven trigger: **Triggers → Add Trigger → Time-driven → Day timer → 9am to 10am**, or `ScriptApp.newTrigger('sendDailySummary').timeBased().atHour(9).everyDays(1).create()`.
23. Read the header row into a dictionary of `name → column index` for each file, then address columns by name. If a branch adds a column, the macro still finds `net_revenue`; if a column is missing, it can say which one instead of silently summing the wrong data.
24. For example: twelve files read; the sum of per-file row counts equals the **Master** row count; and the quarter's total within 25% of the previous run stored on `History`. Each check writes PASS or FAIL with its numbers to a log sheet, and any FAIL stops the email.
25. Power Query wins on effort and maintainability (a few clicks, no macro security, refreshable) and can't do the PDF, the email, or the formatting. The macro wins on those and loses on everything else. The realistic answer for many teams is both: Power Query to build **Master**, a short macro to format, export, and send.
26. Exported `.bas` files are plain text, so Git shows exactly which lines changed; the `.xlsm` is a binary blob where Git can only say "the file changed". Commit the exports, keep the workbook as a build artifact, and note the version in a `Config` cell.
27. First, **make it safe**: take a copy, and find out who receives it and what they do with it. Second, **make it visible**: run it once yourself in a test copy of the folder with `.Display` instead of `.Send`, and read the log; add `Debug.Print` lines if there are none. Third, **make it explainable**: write the one-page note (inputs, outputs, schedule, failure modes), then decide whether to refactor or rewrite. Don't start by rewriting it; start by being able to describe it.
28. When it works, the owner understands it, the data and audience are already in Excel, and the rewrite would buy nothing but elegance. A working automation with a documented owner and a check that stops it when the data is wrong is a good outcome, whatever language it's in. Rewrite when it's fragile, unreadable, unowned, or needs data and scheduling that a spreadsheet can't reach.

Timed challenge answers. Level 1: 12 .xlsx files; *.xls* matches the same twelve here, but would also pick up .xlsb and legacy .xls files, which is a reason to be specific. Level 2: 25,832 rows. Level 3: 25,832 data rows plus one header. Level 4: ₹423,872,808.00. Level 5: Mumbai HO ₹151,058,519.25 · Bengaluru ₹117,726,527.00 · Delhi ₹102,966,828.00 · Kolkata ₹52,120,933.75. Level 6: Oct ₹180,620,103.00 · Nov ₹155,985,902.00 · Dec ₹87,266,803.75. Level 7: seconds versus effectively instant. Bonus: If filesRead <> 12 Then MsgBox "Expected 12 files, found " & filesRead: Exit Sub.